

This problem requires a higher-than-average attention to detail, as there are several non-trivial cases that are easily overlooked. Which is a polite way of saying that while the idea isn't that hard, the implementation can get pretty annoying.

We certainly recommend starting with a brute force solution that's conceptually as simple as possible. Once you get the easy subproblem accepted, you can use the outputs to test your more efficient solutions – and even better, use the brute force solution to generate small counterexamples to your faster solutions. It's pretty likely that your first attempt will miss some of the tricky cases, and having tiny counterexamples helps debug and understand.

There are multiple approaches that will be efficient enough to solve the medium-sized test case. For instance, we can iterate over all pairs (white rook, black rook) and in constant time count the number of ways to move this specific white rook followed by this specific black rook. In terms of the number of cases we need to consider there isn't much of a difference between these solutions and the ones fast enough to solve the hard test case, so we'll go straight to that one.

Parallel movement

We can start by splitting the counting into two cases: either the black rook moves parallel with the white rook, or it moves orthogonally.

The former is easier to count, so let's start with that. We can simplify the counting somewhat by considering some symmetries. In our reference solution we go through four rotations of the board and for each of them we count the ways in which a white rook moves left and then a black rook moves horizontally.

Observe that moving a single white rook always affects only a constant number of black rooks. For almost all black rooks the options they have after the white rook's move will be exactly the same as they are now. Thus, we can start by counting all possible black moves in the initial configuration. We will then solve the rest of the task by iterating over all white rooks, one at a time, and for each of them we'll determine how it affects the nearby black rooks – and thus how to adjust the total number of possible black moves after this particular white rook moves.

As we are considering situations where both rooks move horizontally, only black rooks in the same row as the current white rook can be affected. Here's the full overview of what changes when a particular white rook W moves left:

- If there's a black rook X to the right of W, we are moving W away from X, increasing the number of possible moves. The total number of ways to move W followed by X can be calculated by summing an arithmetic series.
- If there's a black rook Y to the left of W, we are moving W towards Y, which decreases the number of Y's possible moves afterwards. Counting works same as above.
- What, you thought that's all? :) Nah, there's one more case. What happens if there's another black rook Z to the left of Y? One final new option is as follows: First W takes Y, and then Z gains a new option to take W. And don't forget that if W takes Y, we have to subtract all of Y's moves.

Orthogonal movement

Again, we'll start by applying some possible symmetries to reduce the number of cases we need to handle. Here we can consider all eight possible ways of rotating and/or flipping the board, and for each of them we'll count the solutions in which a white rook goes left and then a black rook goes up.

We can precompute some useful information: for each rook, what is the next rook in each direction (if any)? This can easily be done quickly – e.g., in $O(n \log n)$ time by sorting the rooks. (As you may have realized, we actually quietly used some of this information already while solving the parallel case efficiently.)

And also as before, we will start by counting the black moves in the original configuration and then for each white rook we'll determine how the count changes depending on where it moves. In general, if we move a white rook, some new possible moves will appear but some other moves will no longer be possible.

The first type of new black moves we may get after moving the white rook W : If there is a black rook A directly below W , previously the longest possible move for A was to take W . After W moves, A will be able to go through that empty cell into higher cells – until it either hits the top of the chessboard or the next rook in that column. (Remember that we need to look at that rook's color to determine whether A can take it or has to stop one cell sooner.) We can count these new black moves in constant time, and multiply them by the number of possible moves our white rook could make.

The second type of new black moves we may get: if W 's longest possible move is to capture a black rook B , and in B 's column the next rook C below B exists and is also black, then we now get a single new pair of moves: W takes B and then C takes W . (And again, remember that if W takes B then all of B 's moves become unavailable.)

Now that we counted all possible additional moves, we only have to subtract moves that are no longer possible after our white rook moves.

The white rook crosses paths of some black rooks. For each of those rooks: if the white rook stops at any other column, the black rook is unaffected. But if the white rook stops in its particular column, the black rook's movement is now restricted – its new highest reachable cell is the cell now containing our white rook. We need an efficient way of counting the total number of moves prevented in this way.

In the reference solution we do this as follows: For each black rook we have an interval of rows it can visit. White rooks that are in these rows may or may not affect it, but white rooks outside these rows definitely don't affect it. We'll process the white rooks on the chessboard from top to bottom, and we will maintain a set of black rooks whose intervals contain the current white rook's row. For each of these black rooks we know the highest reachable row. We will store these numbers in a tree that will allow us to compute segment sums efficiently (e.g., a Fenwick tree or an interval tree). Now, in order to process a white rook we can identify the segment of "active" black rooks that actually cross the white rook's path (this can be done in logarithmic time) and then we have all we need to count the prevented moves: the number of these black rooks and the sum of their highest reachable rows.

(More precise implementation details: In the reference solution we start by doing coordinate compression on columns – i.e., we collect the numbers of columns that contain at least one black rook. On top of this array we then create two Fenwick trees. One with just 0s and 1s to keep track of which columns contain active black rooks – to be able to find their count in any interval quickly. The other tree is the one described above, with zeros for columns that aren't active and coordinates of top cells for active rooks.)

For example, suppose that our white rook in row 10 crosses the paths of three black rooks: rook X that can currently go up to row 6, rook Y that can only go to row 10, and rook Z that can go all the way up to row 0. If our white rook stops in rook X's column, it will prevent it from moving to rows 6, 7, 8, 9. If it stops in rook Y's column, it won't prevent anything (just the move to an empty square in row 10 will now become a capture). And if our rook stops in rook Z's column, it will prevent 10 of its possible moves. Thus, the total number of prevented moves is $(10-6) + (10-10) + (10-0)$, which can be rewritten as $3 * 10 - (6+10+0)$ and then generalized as "(white rook's row number) times (number of black rooks) minus (sum of black rooks' top cells)".

The solution described above can be implemented to run in $O(n \log n)$ time.